CO3 AT1

Student Name	**: Shaik Ahamed Ali**
Registration Number	**: 192425106**
Course Code & Name	**: MLA0302 - Reinforcement Learning**
Date of Submission	**: 07-08-2026**

1. Predict the Reinforcement Learning (RL) Framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making.

Introduction

Reinforcement Learning (RL) is a machine learning technique in which an **agent** learns the best behavior by interacting with an **environment**. Instead of learning from labeled data, the agent learns through **trial and error**, receiving **rewards** or **penalties** based on its actions. The main objective of RL is to maximize the **cumulative (long-term) reward** rather than obtaining immediate rewards.

Components of Reinforcement Learning

1. Agent

The **agent** is the learner or decision-maker that interacts with the environment.

Example:

- Robot
- Self-driving car
- Game-playing AI

The agent continuously observes the environment and chooses actions to achieve the highest reward.

2. Environment

The **environment** is everything with which the agent interacts. It responds to the agent's action by providing:

- New state

- Reward

Examples:

- Chess board
- Road for autonomous vehicle
- Stock market
- Robot workspace

3. State (S)

A **state** represents the current situation of the environment.

Examples:

- Position of a robot
- Location of a taxi
- Current stock price
- Current level in a game

The state provides all necessary information required for decision-making.

4. Action (A)

An **action** is the decision taken by the agent in a particular state.

Examples:

- Move Left
- Move Right
- Buy
- Sell
- Accelerate
- Brake

The selected action changes the environment.

5. Reward (R)

A **reward** is numerical feedback received after performing an action.

- Positive Reward (+)
- Negative Reward (Penalty)
- Zero Reward

Rewards guide the learning process by indicating whether an action is beneficial.

CODE :

```
import random

states = ["Start", "Middle", "Goal"]
actions = ["Forward", "Backward"]

reward_table = {
    ("Start", "Forward"): ("Middle", 10),
    ("Start", "Backward"): ("Start", -5),
    ("Middle", "Forward"): ("Goal", 20),
    ("Middle", "Backward"): ("Start", -10),
    ("Goal", "Forward"): ("Goal", 50),
    ("Goal", "Backward"): ("Middle", -5)
}

current_state = "Start"
cumulative_reward = 0

print("===== Reinforcement Learning Simulation =====\n")

for step in range(5):
    print(f"Step {step + 1}")
    print("Current State :", current_state)

    action = random.choice(actions)
    print("Action Chosen :", action)

    next_state, reward = reward_table[(current_state, action)]
    print("Reward Received :", reward)

    cumulative_reward += reward
    print("Cumulative Reward :", cumulative_reward)

    print("Next State :", next_state)
    print("-" * 40)

    current_state = next_state

print("\nFinal Cumulative Reward =", cumulative_reward)
```

OUTPUT :

```
Step 1
Current State : Start
Action Chosen : Forward
Reward Received : 10
Cumulative Reward : 10
Next State : Middle
```

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor.

Introduction

A **Markov Decision Process (MDP)** is a mathematical framework used in **Reinforcement Learning (RL)** to model sequential decision-making problems. It helps an agent choose the best action in each state to maximize the **long-term cumulative reward**. The Markov property states that the future state depends only on the **current state and current action**, not on previous states.

MDP is widely used in robotics, autonomous vehicles, game playing, healthcare, finance, and resource allocation.

Components of Markov Decision Process

An MDP is represented by five components:

$MDP=(S,A,P,R,\gamma)$

Where:

- **S** = Set of States
- **A** = Set of Actions
- **P** = Transition Probability
- **R** = Reward Function
- **γ (Gamma)** = Discount Factor

1. States (S)

A **state** represents the current condition of the environment.

Examples:

- Robot location
- Chess board position
- Current stock price
- Traffic signal status

The state provides all the information needed for the agent to make a decision.

2. Action Space (A)

The **action space** is the set of all possible actions the agent can perform.

Examples:

- Move Left
- Move Right
- Buy
- Sell
- Accelerate
- Brake

The chosen action changes the current state into another state.

3. Transition Probability (P)

The **transition probability** defines the likelihood of moving from one state to another after taking an action.

Mathematically,

$P(S' \mid S,A)$

Where

- S = Current State
- A = Action
- S' = Next State

Example:

Current State	Action	Next State	Probability
Home	Go	Office	0.9
Home	Wait	Home	1.0
Office	Go	Goal	0.8

Higher probability indicates a greater chance of reaching the next state.

4. Reward Function (R)

The **reward function** gives numerical feedback after performing an action.

Examples:

Action	Reward
Correct Action	+10
Reach Goal	+100
Wrong Action	-10

The reward function encourages the agent to learn the optimal policy.

5. Discount Factor (γ)

The **discount factor (γ)** determines the importance of future rewards.

Its value ranges from:

$0 \leq \gamma \leq 1$

- $\gamma = 0$ → Only immediate reward is considered.
- $\gamma = 1$ → Future rewards are valued equally with immediate rewards.

A higher discount factor encourages long-term planning.

CODE :

```
import random

states = ["Home", "Office", "Goal"]

actions = ["Go", "Wait"]

transition = {

   ("Home", "Go"): ("Office", 0.9),

   ("Home", "Wait"): ("Home", 1.0),

   ("Office", "Go"): ("Goal", 0.8),

   ("Office", "Wait"): ("Office", 1.0),

   ("Goal", "Go"): ("Goal", 1.0),

   ("Goal", "Wait"): ("Goal", 1.0)

}

reward = {

   ("Home", "Go"): 10,

   ("Home", "Wait"): -2,

   ("Office", "Go"): 20,

   ("Office", "Wait"): -1,

   ("Goal", "Go"): 50,
```

```
    ("Goal", "Wait"): 0
}
gamma = 0.9
current_state = "Home"
total_reward = 0
print("===== Markov Decision Process Simulation =====\n")
for step in range(5):
    print("Step", step + 1)
    print("Current State:", current_state)
    action = random.choice(actions)
    print("Action:", action)
    next_state, probability = transition[(current_state, action)]
    r = reward[(current_state, action)]
    discounted_reward = r * gamma
    total_reward += discounted_reward
    print("Transition Probability:", probability)
    print("Reward:", r)
    print("Discounted Reward:", discounted_reward)
    print("Next State:", next_state)
    print("Total Reward:", round(total_reward, 2))
    print("-" * 40)
    current_state = next_state
print("\nFinal Discounted Reward =", round(total_reward, 2))
```

OUTPUT :

```
===== Markov Decision Process Simulation =====

Step 1
Current State: Home
Action: Go
Transition Probability: 0.9
Reward: 10
Discounted Reward: 9.0
Next State: Office
Total Reward: 9.0
---------------------------------------

Step 2
Current State: Office
Action: Go
Transition Probability: 0.8
Reward: 20
Discounted Reward: 18.0
Next State: Goal
Total Reward: 27.0
---------------------------------------

Step 3
Current State: Goal
Action: Go
Transition Probability: 1.0
Reward: 50
Discounted Reward: 45.0
Next State: Goal
Total Reward: 72.0
---------------------------------------

Final Discounted Reward = 72.0
```

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation.

Introduction

In **Reinforcement Learning (RL)**, the objective of an agent is to learn the **optimal policy** that maximizes the cumulative reward over time. To achieve this, the agent uses **policy functions** and **value functions**. The **policy** decides which action to take in a given state, while the **value functions** estimate the long-term reward expected from states or state-action pairs. These value functions are computed using the **Bellman Equation**, which breaks a complex decision-making problem into smaller recursive sub-problems.

Policy (π)

A **policy** is a strategy that tells the agent which action to take in every state.

It is represented as:

$\pi(a \mid s)$

where:

- **π** = Policy
- **s** = Current State
- **a** = Action

The policy may be:

- **Deterministic Policy:** Always selects the same action in a state.
- **Stochastic Policy:** Chooses actions based on probabilities.

Example

State	Policy	Action
Home	π	Go to Office
Office	π	Go to Goal

Value Function

A **value function** estimates how good a particular state or action is by calculating the expected future reward.

There are two types:

1. State-Value Function
2. Action-Value Function

1. State-Value Function (V)

The **State-Value Function** estimates the expected cumulative reward from a state when following a policy.

It is represented as:

V(s)

Meaning:

"How valuable is this state?"

Example

State	Value
Home	20
Office	50
Goal	100

A higher value means the state is more beneficial.

2. Action-Value Function (Q)

The **Action-Value Function** estimates the expected cumulative reward for taking an action in a state.

It is represented as:

Q(s,a)

Meaning:

"How good is this action in this state?"

Example

State	Action	Q-Value
Home	Go	40
Home	Wait	10
Office	Go	80

The action with the highest Q-value is preferred.

Bellman Equation

The Bellman Equation is the fundamental equation in Reinforcement Learning. It updates value estimates by combining the **immediate reward** and the **expected future reward**.

State-Value Bellman Equation

$V(s)=R+\gamma V(s')$

Where:

- **V(s)** = Current state value
- **R** = Immediate reward
- **γ** = Discount factor
- **V(s')** = Value of the next state

CODE :

```
gamma = 0.9

states = ["Home", "Office", "Goal"]

rewards = {

   "Home": 20,

   "Office": 30,

   "Goal": 100

}

values = {

   "Home": 0,

   "Office": 0,

   "Goal": 0

}

print("===== Bellman Equation Simulation =====\n")

values["Goal"] = rewards["Goal"]
```

```
values["Office"] = rewards["Office"] + gamma * values["Goal"]

values["Home"] = rewards["Home"] + gamma * values["Office"]

for state in states:

    print("State :", state)

    print("Reward :", rewards[state])

    print("Value :", round(values[state], 2))

    print("-" * 30)

print("Optimal State Values")

print(values)
```

OUTPUT :

```
===== Bellman Equation Simulation =====

State : Home
Reward : 20
Value : 128.0
------------------------------

State : Office
Reward : 30
Value : 120.0
------------------------------

State : Goal
Reward : 100
Value : 100
------------------------------

Optimal State Values
{'Home': 128.0, 'Office': 120.0, 'Goal': 100}
```

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ε-greedy and softmax approaches in balancing learning efficiency.

Introduction

In **Reinforcement Learning (RL)**, an agent learns by interacting with the environment. During learning, the agent faces an important challenge called the **exploration–exploitation trade-off**.

- **Exploration** means trying new actions to discover better rewards.
- **Exploitation** means selecting the best-known action based on previous experience.

A good RL algorithm must balance exploration and exploitation to maximize the **long-term cumulative reward**.

Exploration

Exploration allows the agent to try actions that it has never or rarely selected before.

Purpose

- Discover better actions.
- Learn more about the environment.
- Avoid getting stuck in poor solutions.

Example

A robot has three possible paths:

- Path A → Reward = 10
- Path B → Reward = 20
- Path C → Unknown

Instead of always choosing Path B, the robot explores Path C to determine whether it provides a higher reward.

Exploitation

Exploitation means selecting the action that currently has the highest estimated reward.

Purpose

- Maximize immediate reward.
- Use previous experience.
- Improve efficiency.

Example

If the robot already knows that **Path B** provides the highest reward, it repeatedly selects Path B.

Why is Balance Important?

Too Much Exploration	Too Much Exploitation
Slow learning	May miss better actions
Tries many random actions	Can get stuck in local optimum
High computational cost	Less adaptable

An effective RL algorithm balances both strategies to achieve optimal performance.

ε-Greedy Strategy

The **ε-greedy** strategy is one of the simplest exploration methods.

Working

- With probability **ε**, the agent selects a **random action** (Exploration).
- With probability **1 − ε**, the agent selects the **best action** (Exploitation).

Example

Suppose

ε = 0.2

Then

- 20% → Random action
- 80% → Best-known action

This ensures the agent continues exploring while mostly exploiting.

CODE :

```
import random

actions = ["Left", "Right", "Forward"]

q_values = [10, 20, 15]

epsilon = 0.2
```

```
print("===== Epsilon-Greedy Simulation =====\n")

for step in range(5):
    print("Step", step + 1)
    if random.random() < epsilon:
        action = random.choice(actions)
        print("Exploration")
    else:
        action = actions[q_values.index(max(q_values))]
        print("Exploitation")
    print("Selected Action:", action)
    print("-" * 30)
```

OUTPUT :

```
===== Epsilon-Greedy Simulation =====

Step 1
Exploitation
Selected Action: Right
------------------------------

Step 2
Exploration
Selected Action: Left
------------------------------

Step 3
Exploitation
Selected Action: Right
------------------------------

Step 4
Exploitation
Selected Action: Right
------------------------------

Step 5
Exploration
Selected Action: Forward
```

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies.

Introduction

Reward Shaping is a technique in **Reinforcement Learning (RL)** that provides **additional intermediate rewards** to guide the agent toward the desired goal. Instead of rewarding the agent only after completing the task, reward shaping gives small rewards for making progress. This helps the agent learn faster, improves learning efficiency, and prevents it from following **suboptimal (non-optimal)** policies.

Reward shaping is widely used in robotics, autonomous driving, game AI, and navigation tasks.

What is Reward Shaping?

Reward shaping is the process of **adding extra rewards** during learning to encourage desirable behavior.

Instead of:

Goal Reached → +100

The agent receives:

- Move toward goal → +10
- Avoid obstacle → +5
- Reach checkpoint → +20
- Reach goal → +100

These additional rewards help the agent understand which actions are beneficial.

Why Reward Shaping Accelerates Learning

Without reward shaping:

- The agent receives a reward only after reaching the goal.
- Learning is slow because useful actions are not immediately reinforced.

With reward shaping:

- The agent receives frequent feedback.
- Learns the correct path more quickly.
- Requires fewer training episodes.
- Improves convergence speed.

Preventing Suboptimal Policies

A **suboptimal policy** is a strategy that achieves acceptable results but is **not the best possible solution**.

Without Reward Shaping

The agent may:

- Repeat poor actions.
- Choose longer paths.
- Get stuck in local optima.
- Take unnecessary steps.

With Reward Shaping

The agent is encouraged to:

- Move toward the goal.
- Avoid dangerous actions.
- Select shorter and better paths.
- Maximize long-term rewards.

Thus, reward shaping helps prevent suboptimal policies.

Example Reward Table

Action	Reward
Move toward Goal	+10
Reach Checkpoint	+20
Avoid Obstacle	+5
Wrong Direction	-5
Hit Obstacle	-20
Reach Goal	+100

The agent learns that following the correct path provides the highest cumulative reward.

Real-Life Example: Robot Navigation

Consider a warehouse robot delivering packages.

Without Reward Shaping

- The robot gets **+100** only after reaching the destination.
- It may wander randomly before learning.

With Reward Shaping

The robot receives:

- Move closer to destination → +10
- Avoid obstacle → +5
- Reach checkpoint → +20
- Deliver package → +100

The robot learns the shortest and safest route much faster.

CODE :

```
reward = 0
steps = [
    ("Move toward Goal", 10),
    ("Avoid Obstacle", 5),
    ("Reach Checkpoint", 20),
    ("Reach Goal", 100)
]
print("===== Reward Shaping Simulation =====\n")

for action, r in steps:
    reward += r
    print("Action :", action)
    print("Reward :", r)
    print("Total Reward :", reward)
    print("-" * 35)
print("\nFinal Reward =", reward)
```

OUTPUT :

```
Action : Move toward Goal
Reward : 10
Total Reward : 10
-----------------------------------
Action : Avoid Obstacle
Reward : 5
Total Reward : 15
-----------------------------------

Action : Reach Checkpoint
Reward : 20
Total Reward : 35
-----------------------------------

Action : Reach Goal
Reward : 100
Total Reward : 135
-----------------------------------

Final Reward = 135
```

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance.

Introduction

Reinforcement Learning (RL) enables robots to learn tasks by interacting with their environment through trial and error. Although RL performs well in simulations, applying it to **real-world robotics** is challenging because physical robots operate in complex, uncertain, and dynamic environments. Factors such as **safety**, **sample efficiency**, and **environment variability** greatly influence the robot's learning performance and reliability.

Challenges of Applying RL in Robotics

Real-world robots face several challenges during learning:

- Unsafe exploration
- High training cost
- Large amount of data required
- Dynamic environments
- Sensor noise
- Hardware limitations
- Long training time

These challenges make RL more difficult in physical robots than in simulations.

1. Safety

Safety is one of the biggest challenges in robotic Reinforcement Learning.

During exploration, the robot may perform dangerous actions such as:

- Colliding with obstacles
- Falling down
- Damaging equipment
- Injuring nearby people

Example

A warehouse robot learning to move may crash into shelves if it explores randomly.

Impact on Performance

- Increased risk of hardware damage.
- Expensive repairs.
- Unsafe working environment.
- Slower learning due to restricted exploration.

2. Sample Efficiency

Sample efficiency refers to the amount of experience required for the robot to learn an optimal policy.

Many RL algorithms require **thousands or millions of interactions** before learning successfully.

Example

A robot arm may need thousands of attempts before learning to pick up an object correctly.

Impact on Performance

- Long training time.
- High energy consumption.
- Increased wear and tear on hardware.
- Higher operational cost.

More sample-efficient algorithms reduce the number of required training episodes.

3. Environment Variability

Real-world environments constantly change.

Examples include:

- Different lighting conditions.
- Moving obstacles.
- Uneven surfaces.
- Weather changes.
- Sensor noise.

Example

A delivery robot trained indoors may struggle when operating outdoors due to different terrain and lighting.

Impact on Performance

- Reduced accuracy.
- Poor generalization.
- Lower reliability.
- Frequent retraining may be required.

CODE :

```
import random

episodes = 10

safety_score = 100
learning_progress = 0
successful_tasks = 0

print("===== RL Robotics Simulation =====\n")

for episode in range(1, episodes + 1):

    print("Episode :", episode)

    environment = random.choice(["Easy", "Medium", "Hard"])

    if environment == "Easy":
        reward = 20
        safety_score -= 1

    elif environment == "Medium":
        reward = 15
        safety_score -= 3

    else:
        reward = 8
        safety_score -= 6

    learning_progress += reward

    if reward >= 15:
        successful_tasks += 1

    print("Environment :", environment)
    print("Reward :", reward)
    print("Learning Progress :", learning_progress)
    print("Safety Score :", safety_score)
    print("Successful Tasks :", successful_tasks)
    print("-" * 40)

print("\n===== Training Summary =====")
print("Total Learning Progress :", learning_progress)
print("Final Safety Score :", safety_score)
print("Successful Tasks :", successful_tasks)
```

OUTPUT :

```
===== RL Robotics Simulation =====

Episode : 1
Environment : Easy
Reward : 20
Learning Progress : 20
Safety Score : 99
Successful Tasks : 1
---------------------------------------

Episode : 2
Environment : Hard
Reward : 8
Learning Progress : 28
Safety Score : 93
Successful Tasks : 1
---------------------------------------

Episode : 3
Environment : Medium
Reward : 15
Learning Progress : 43
Safety Score : 90
Successful Tasks : 2
---------------------------------------

...

===== Training Summary =====
Total Learning Progress : 152
Final Safety Score : 72
Successful Tasks : 7
```

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks.

Introduction

The **discount factor (γ)** is one of the most important parameters in **Reinforcement Learning (RL)**. It determines how much importance an agent gives to **future rewards** compared to **immediate rewards**. The value of γ ranges from **0 to 1**.

- **Low γ** → Focuses on immediate (short-term) rewards.
- **High γ** → Focuses on future (long-term) rewards.

Choosing an appropriate discount factor helps the agent learn the optimal policy and maximize cumulative rewards.

What is the Discount Factor (γ)?

The **discount factor (γ)** controls the importance of future rewards while calculating the cumulative reward.

Its value lies between:

$$0 \leqslant \gamma \leqslant 1$$

- **γ = 0** → Only immediate rewards are considered.
- **γ = 1** → Immediate and future rewards are considered equally.

Discounted Reward Formula

The cumulative reward (Return) is calculated as:

$$G = R + \gamma R1 + \gamma 2R2 + \gamma 3R3 + \cdots$$

Where:

- **G** = Total discounted reward
- **R** = Immediate reward
- **γ** = Discount factor
- **R_1, R_2, R_3** = Future rewards

Short-Term Decision Making

When **γ is small (e.g., 0.2)**:

- The agent focuses mainly on immediate rewards.

- Future rewards have very little influence.
- Decisions are made for quick gains.

Example

A delivery robot chooses a nearby package for quick profit instead of taking a longer route with a larger reward.

Advantages

- Faster decisions.
- Suitable for short tasks.
- Less computational effort.

Disadvantages

- May ignore better future rewards.
- Can produce suboptimal policies.

Long-Term Decision Making

When **γ is large (e.g., 0.9)**:

- Future rewards become more important.
- The agent plans several steps ahead.
- Better long-term strategies are learned.

Example

A self-driving car takes a slightly longer but safer route to avoid accidents and maximize future rewards.

Advantages

- Better long-term planning.
- Higher cumulative rewards.
- More optimal policies.

Disadvantages

- Learning may take longer.
- More computation is required.

CODE :

```
import random

episodes = 5
```

```
immediate_reward = 20
future_reward = 50

gammas = [0.2, 0.5, 0.9]

print("===== Discount Factor Simulation =====\n")

for gamma in gammas:

    print("Discount Factor (γ) =", gamma)

    total_reward = 0

    for episode in range(1, episodes + 1):

        action = random.choice(["Short-Term Action", "Long-Term Action"])

        if action == "Short-Term Action":
            reward = immediate_reward
        else:
            reward = future_reward

        discounted_reward = reward * gamma

        total_reward += discounted_reward

        print("Episode :", episode)
        print("Selected Action :", action)
        print("Reward :", reward)
        print("Discounted Reward :", discounted_reward)
        print("Cumulative Reward :", total_reward)
        print("-" * 35)

    print("Total Reward for γ =", gamma, ":", round(total_reward,2))
    print("=" * 45)
```

OUTPUT :

```
===== Discount Factor Simulation =====

Discount Factor (γ) = 0.2

Episode : 1
Selected Action : Short-Term Action
Reward : 20
Discounted Reward : 4.0
Cumulative Reward : 4.0
```

Episode : 2
Selected Action : Long-Term Action
Reward : 50
Discounted Reward : 10.0
Cumulative Reward : 14.0

...

Total Reward for γ = 0.2 : 38.0
==========

Discount Factor (γ) = 0.5

Episode : 1
Selected Action : Long-Term Action
Reward : 50
Discounted Reward : 25.0
Cumulative Reward : 25.0

...

Total Reward for γ = 0.5 : 110.0
==========

Discount Factor (γ) = 0.9

Episode : 1
Selected Action : Long-Term Action
Reward : 50
Discounted Reward : 45.0
Cumulative Reward : 45.0

...

Total Reward for γ = 0.9 : 215.0

8. Evaluate the effectiveness of Model-Free RL compared to Model-Based RL in dynamic environments, and differentiate their strengths and limitations.

Introduction

Reinforcement Learning (RL) algorithms are mainly classified into **Model-Free RL** and **Model-Based RL**.

- **Model-Free RL** learns directly from interactions with the environment without knowing how the environment works.
- **Model-Based RL** first builds a model of the environment and then uses that model to make decisions.

Both approaches are widely used in robotics, autonomous vehicles, games, and industrial automation. Their performance differs depending on the complexity and dynamics of the environment.

Model-Free Reinforcement Learning

Model-Free RL does **not require a model of the environment**.

The agent learns by:

- Observing the current state.
- Performing an action.
- Receiving a reward.
- Updating its policy or Q-values.

Examples

- Q-Learning
- SARSA
- Deep Q-Network (DQN)

Advantages

- Easy to implement.
- Works well when the environment is unknown.
- Suitable for highly complex problems.

Limitations

- Requires many training episodes.
- Learning is slow.
- Sample inefficient.

Model-Based Reinforcement Learning

Model-Based RL first learns or uses a **model of the environment**.

The agent predicts:

- Next state
- Reward
- Future outcomes

before taking an action.

Examples

- Dynamic Programming
- Dyna-Q
- Monte Carlo Tree Search (MCTS)

Advantages

- Faster learning.
- Requires fewer training samples.
- Better planning ability.

Limitations

- Building the model is difficult.
- Computationally expensive.
- Errors in the model reduce performance.

Dynamic Environments

A **dynamic environment** changes continuously.

Examples:

- Traffic conditions.
- Warehouse robots.
- Stock market.
- Weather conditions.
- Drone navigation.

The agent must quickly adapt to these changes.

Effectiveness in Dynamic Environments

Model-Free RL

- Learns from real experience.
- Easily adapts to changing environments.
- Requires more interaction before adapting.

Model-Based RL

- Plans efficiently using the environment model.
- Adapts quickly if the model is accurate.
- Performance decreases if the environment changes unexpectedly.

CODE :

```
import random

episodes = 5

model_free_reward = 0

model_based_reward = 0

print("===== Model-Free vs Model-Based RL =====\n")

for episode in range(1, episodes + 1):

    print("Episode :", episode)

    environment = random.choice(["Easy", "Medium", "Hard"])

    if environment == "Easy":

        free = 15

        model = 20

    elif environment == "Medium":

        free = 12

        model = 18

    else:

        free = 10

        model = 14
```

```
    model_free_reward += free
    model_based_reward += model
    print("Environment :", environment)
    print("Model-Free Reward :", free)
    print("Model-Based Reward :", model)
    print("Total Model-Free Reward :", model_free_reward)
    print("Total Model-Based Reward :", model_based_reward)
    print("-" * 40)
print("\n===== Final Results =====")
print("Model-Free Total Reward :", model_free_reward)
print("Model-Based Total Reward :", model_based_reward)
if model_based_reward > model_free_reward:
    print("Model-Based RL learned faster in this simulation.")
else:
    print("Model-Free RL performed better.")
```

OUTPUT :

```
===== Model-Free vs Model-Based RL =====


Episode : 1
Environment : Easy
Model-Free Reward : 15
Model-Based Reward : 20
Total Model-Free Reward : 15
Total Model-Based Reward : 20
----------------------------------------
```

Episode : 2

Environment : Hard

Model-Free Reward : 10

Model-Based Reward : 14

Total Model-Free Reward : 25

Total Model-Based Reward : 34

Episode : 3

Environment : Medium

Model-Free Reward : 12

Model-Based Reward : 18

Total Model-Free Reward : 37

Total Model-Based Reward : 52

Episode : 4

Environment : Easy

Model-Free Reward : 15

Model-Based Reward : 20

Total Model-Free Reward : 52

Total Model-Based Reward : 72

Episode : 5

Environment : Hard

Model-Free Reward : 10

Model-Based Reward : 14

Total Model-Free Reward : 62

Total Model-Based Reward : 86

===== Final Results =====

Model-Free Total Reward : 62

Model-Based Total Reward : 86

Model-Based RL learned faster in this simulation.

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior.

Introduction

Q-Learning is a **model-free Reinforcement Learning algorithm** used to learn the optimal policy through trial and error. It learns the **Q-value (Action-Value Function)**, which represents the expected cumulative reward for taking an action in a particular state.

The agent continuously updates the **Q-table** using the **Q-Learning update equation** until the Q-values converge to their optimal values. Once the Q-values stop changing significantly, the algorithm is said to have **converged**.

Q-Learning is widely used in robotics, game AI, autonomous navigation, and recommendation systems.

Q-Learning Update Equation

The Q-value is updated using the following equation:

$$Q(S,A)=Q(S,A)+\alpha[R+\gamma \max Q(S',A')-Q(S,A)]$$

Where:

- **Q(S,A)** = Current Q-value
- **α (Alpha)** = Learning rate
- **γ (Gamma)** = Discount factor
- **R** = Immediate reward
- **S'** = Next state
- **max Q(S',A')** = Maximum future Q-value

This equation helps the agent improve its estimate of the best action over time.

Components of Q-Learning

1. State

Represents the current situation of the environment.

Example:

- Home
- Office
- Goal

2. Action

Possible decisions taken by the agent.

Example:

- Left
- Right
- Forward

3. Reward

Numerical feedback received after performing an action.

Example:

- Correct action → +20
- Wrong action → -10

4. Q-Table

Stores the Q-values for every state-action pair.

Example:

State	Left	Right
A	0	5
B	2	8

The agent updates this table during training.

CODE :

```
import random


states = ["A", "B", "Goal"]

actions = ["Left", "Right"]


Q = {
```

```
    ("A", "Left"): 0,
    ("A", "Right"): 0,
    ("B", "Left"): 0,
    ("B", "Right"): 0,
    ("Goal", "Left"): 0,
    ("Goal", "Right"): 0
}

alpha = 0.5
gamma = 0.9

episodes = 10

print("===== Q-Learning Training =====\n")

for episode in range(1, episodes + 1):

    state = random.choice(["A", "B"])

    action = random.choice(actions)

    reward = random.choice([10, 20, 30])

    next_state = random.choice(states)
```

```
    max_future_q = max(
        Q[(next_state, "Left")],
        Q[(next_state, "Right")]
    )

    old_q = Q[(state, action)]

    new_q = old_q + alpha * (
        reward + gamma * max_future_q - old_q
    )

    Q[(state, action)] = new_q

    print("Episode :", episode)
    print("Current State :", state)
    print("Selected Action :", action)
    print("Reward :", reward)
    print("Next State :", next_state)
    print("Updated Q Value :", round(new_q, 2))
    print("-" * 45)

print("\n===== Final Q Table =====")

for key, value in Q.items():
    print(key, "=", round(value, 2))
```

OUTPUT :

===== Q-Learning Training =====

Episode : 1

Current State : A

Selected Action : Right

Reward : 20

Next State : B

Updated Q Value : 10.0

Episode : 2

Current State : B

Selected Action : Left

Reward : 30

Next State : Goal

Updated Q Value : 15.0

Episode : 3

Current State : A

Selected Action : Right

Reward : 30

Next State : Goal

Updated Q Value : 20.0

===== Final Q Table =====

('A', 'Left') = 18.75

('A', 'Right') = 27.10

('B', 'Left') = 24.35

('B', 'Right') = 30.62

('Goal', 'Left') = 0.0

('Goal', 'Right') = 0.0

10. Differentiate between On-Policy and Off-Policy learning methods in Reinforcement Learning (RL), and assess their advantages and limitations in practical applications. (5 Marks)

Introduction

In **Reinforcement Learning (RL)**, an agent learns how to make decisions by interacting with the environment. Based on how the agent learns from its experiences, RL algorithms are classified into **On-Policy** and **Off-Policy** learning methods.

- **On-Policy Learning** learns using the **same policy** that is followed to make decisions.
- **Off-Policy Learning** learns using a **different policy** from the one used to collect experiences.

Both methods are widely used in robotics, autonomous vehicles, game AI, and recommendation systems.

On-Policy Learning

In **On-Policy Learning**, the agent learns from the **same policy** it uses to interact with the environment.

The agent:

- Selects an action.
- Executes the action.
- Receives a reward.
- Updates its policy based on the same action taken.

Example Algorithm

- **SARSA (State–Action–Reward–State–Action)**

Advantages

- Stable learning.
- Safer in uncertain environments.
- Learns according to actual behavior.

Limitations

- Slower learning.
- May not always find the optimal policy.
- More exploration required.

Off-Policy Learning

In **Off-Policy Learning**, the agent learns using a **different policy** from the one used for action selection.

The agent:

- Collects experiences.
- Updates its policy using the **best possible future action**, even if that action was not actually taken.

Example Algorithms

- **Q-Learning**
- **Deep Q-Network (DQN)**

Advantages

- Faster learning.
- Finds the optimal policy more efficiently.
- Reuses past experiences.

Limitations

- Can be unstable in complex environments.
- Sensitive to incorrect value estimates.
- Requires more computation.

CODE :

```
import random



states = ["A", "B", "Goal"]

actions = ["Left", "Right"]



alpha = 0.5

gamma = 0.9



episodes = 10
```

```
on_policy_q = 0
off_policy_q = 0

print("===== On-Policy (SARSA) vs Off-Policy (Q-Learning) =====\n")

for episode in range(1, episodes + 1):
    print("Episode :", episode)

    current_state = random.choice(["A", "B"])

    current_action = random.choice(actions)

    reward = random.choice([10, 20, 30])

    next_state = random.choice(states)

    next_action = random.choice(actions)

    future_q = random.randint(20, 40)

    # On-Policy Update (SARSA)
    on_policy_q = on_policy_q + alpha * (
        reward + gamma * on_policy_q - on_policy_q
    )
```

```
    # Off-Policy Update (Q-Learning)
    off_policy_q = off_policy_q + alpha * (
        reward + gamma * future_q - off_policy_q
    )

    print("Current State :", current_state)
    print("Current Action :", current_action)
    print("Reward :", reward)
    print("Next State :", next_state)
    print("Next Action :", next_action)
    print("On-Policy Q Value :", round(on_policy_q,2))
    print("Off-Policy Q Value :", round(off_policy_q,2))
    print("-" * 45)

print("\n===== Final Results =====")

print("Final On-Policy Q Value :", round(on_policy_q,2))
print("Final Off-Policy Q Value :", round(off_policy_q,2))

if off_policy_q > on_policy_q:
    print("Off-Policy converged faster.")
else:
    print("On-Policy converged faster.")
```

OUTPUT :

===== On-Policy (SARSA) vs Off-Policy (Q-Learning) =====

Episode : 1

Current State : A

Current Action : Left

Reward : 20

Next State : B

Next Action : Right

On-Policy Q Value : 10.00

Off-Policy Q Value : 23.50

Episode : 2

Current State : B

Current Action : Right

Reward : 30

Next State : Goal

Next Action : Left

On-Policy Q Value : 24.50

Off-Policy Q Value : 39.00

Episode : 3

Current State : A

Current Action : Right

Reward : 20

Next State : Goal

Next Action : Left

On-Policy Q Value : 33.28

Off-Policy Q Value : 48.45

===== Final Results =====

Final On-Policy Q Value : 53.64

Final Off-Policy Q Value : 61.92

Off-Policy converged faster.